

CareerAtlas

AI-Powered Personal Skill Gap Intelligence System

Capstone Project Proposal • GenAI Workshop • March 2026

Submitted by: The Transformers

Members: Tanush Sudheer Tambe • Dripto Bhattacharya • Shivanshu Gupta • Sidhaarth Shree • G Hansini

1. Problem Statement

Students and early-career professionals face three compounding structural hurdles when attempting to upskill or transition into technical roles:

- **Role Ambiguity:** Lack of clarity on precise, industry-standard skill requirements for specific roles, leaving candidates unable to target preparation effectively.
- **Unidentified Knowledge Gaps:** Inability to benchmark current competencies against live industry demands, producing blind spots that only surface during interviews or on the job.
- **Resource Fragmentation:** Learning materials scattered across MOOCs, documentation & YouTube offer conflicting roadmaps, resulting in redundant coursework or missed fundamentals.

Existing solutions - *LinkedIn Learning, Coursera career paths, generic roadmaps* - are static and one-size-fits-all. No intelligent system currently takes a user's actual demonstrated knowledge (resume and structured inputs) and generates a personalized, dependency-aware, actionable, market-aligned roadmap tied to real job market requirements + opportunities. **CareerAtlas** addresses this gap by combining structured profile extraction, retrieval-based analysis & intelligent planning to deliver personalized, actionable career pathways.

2. Proposed Solution & Core Features

CareerAtlas is an end-to-end, agent-driven system that performs structured profile extraction, retrieval-grounded skill gap analysis and generates personalized, market-aligned career roadmaps. Instead of relying on static or manually curated skill ontologies, the system anchors all reasoning in a continuously updated retrieval layer built on vectorized job requirements and curated learning resources. This ensures that outputs remain aligned with real-world industry expectations while adapting to each user's current skill profile.

2.1 Core Features

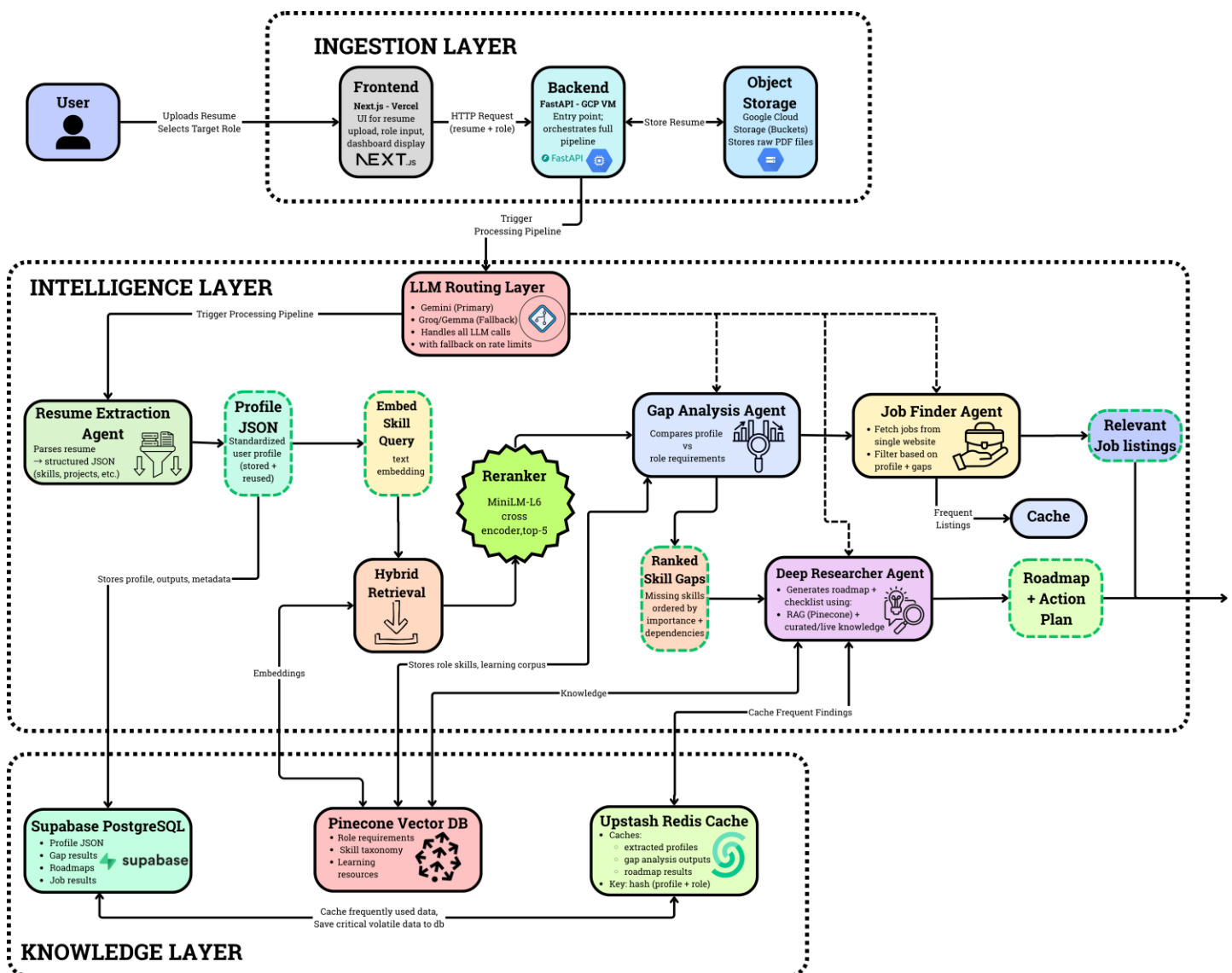
- ✚ **Structured Profile Extraction:** Resume PDFs are parsed using Gemini (with fallback LLM routing) into a strict JSON schema capturing skills, education, projects, and signals. The extraction process is deterministic and schema-constrained to ensure downstream reliability.
- ✚ **RAG-Based Skill Gap Analysis:** The extracted profile is compared against role-specific requirements stored in Pinecone. Semantic retrieval identifies missing competencies and ranks them based on relevance and dependency order, producing a structured gap profile.

- Deep Researcher Agent (Primary Intelligence Layer):** A dedicated agent synthesizes roadmap and action plans using a combination of retrieved knowledge (courses, documentation, role expectations) and curated/live research. It generates step-by-step learning paths, project suggestions, and actionable checklists aligned with current market trends.
- Job Matching Agent:** The system can integrate a job discovery component that maps the user's current profile and gap status to relevant job postings from public sources.

3. System Architecture & Data Flow

3.1 Four-Layer System Design [\(Link to High Resolution Diagram\)](#)

The system follows a modular agent architecture where each stage - extraction, gap analysis and research, operates independently but is connected through structured data contracts. Retrieval-Augmented Generation (RAG) is used selectively to ground decisions in relevant knowledge rather than relying purely on model reasoning, improving both accuracy and consistency.



3.2 Multi-Agent Workflow (LangGraph)

The LangGraph supervisor orchestrates a set of specialized agents in a directed workflow with conditional routing and fallback edges. Each agent operates on structured inputs and produces deterministic outputs that feed into downstream stages. The system is designed to be modular, allowing optional agents (e.g., job matching) to be included without affecting the core pipeline.

LLM routing is integrated at the orchestration layer, where Gemini acts as the primary model and alternative providers (e.g., Groq / Gemma) are used as fallback under rate limits or failures.

<i>Agent</i>	<i>Input</i>	<i>Tool / Model</i>	<i>Output</i>
<i>Extraction Agent</i>	Resume PDF	Gemini 3 Flash (primary) + fallback LLMs; schema-constrained prompting	Structured profile JSON {skills, education, projects, signals, evidence}
<i>Gap Analysis Agent</i>	Profile JSON + target role	Pinecone semantic retrieval (RAG)	Ranked skill gaps with prerequisite ordering and relevance scores
<i>Deep Researcher Agent</i>	Profile JSON + ranked gaps	RAG (Pinecone) + LLM synthesis + optional live/curated search	Personalized roadmap, project plan, and actionable checklist
<i>Job Finder Agent</i>	Profile JSON + profile gaps + target role	Public job APIs / scraping + LLM ranking	Filtered and ranked job opportunities aligned with current profile

4. Technology Stack

<i>Component</i>	<i>Technology</i>	<i>Free Tier Limit / Notes</i>
<i>LLM & Reasoning (Primary)</i>	Gemini 3.1 Flash-Lite	~500 requests/day • 15 RPM • high context • suitable for structured extraction & reasoning
<i>LLM Fallback Layer</i>	Groq (LLaMA 3.x / Mistral) • Gemma 3 (via API)	High RPM/TPM limits • used on rate-limit/failure fallback • faster inference for lightweight tasks
<i>Agent Orchestration</i>	LangGraph (Python)	Open source • manages multi-agent workflow with conditional routing
<i>Backend Framework</i>	FastAPI (Python)	Open source • lightweight, async-friendly API
<i>Vector Database (RAG)</i>	Pinecone	Starter tier: ~2 GB storage • 1 index • sufficient for MVP retrieval
<i>Relational Database</i>	Supabase PostgreSQL	500 MB storage • 50,000 rows free • supports JSONB for semi-structured data
<i>Response Cache</i>	Upstash Redis	~10K commands/day free • used for caching extraction, gaps, and roadmap outputs
<i>Object Storage</i>	Google Cloud Storage (GCS)	~5 GB free tier • integrated with GCP backend • used for storing resume PDFs
<i>Authentication</i>	Supabase Auth	Free tier with email/password + OAuth • simple integration with frontend/backend
<i>Frontend</i>	Next.js 15/16	Open source • server components + API routes support
<i>Backend Hosting</i>	Google Cloud Run (or GCP VM for dev)	~2M requests/month free • auto-scaling serverless deployment
<i>Frontend Hosting</i>	Vercel (Hobby Tier)	~100 GB bandwidth/month free • seamless Next.js deployment
<i>Monitoring & Logging</i>	Sentry + GCP Logging	~5K errors/month free • basic observability and debugging
<i>CI/CD</i>	GitHub Actions	~2,000 minutes/month free • automated build and deployment pipelines

5. Cost Analysis

Service	Development Phase Usage	Dev Cost (₹/month)	Production Phase Usage (~500 users)	Prod Cost (₹/month)
<i>LLM (Gemini 3.1 Flash-Lite + fallback)</i>	Within free-tier limits (low-volume testing)	₹ 0.00	~20K–40K requests/month (with fallback usage)	~₹1,000–1,800
<i>Supabase PostgreSQL</i>	~5K rows, <100 MB data (free tier)	₹ 0.00	Supabase Pro (8 GB DB, higher limits)	~₹2,300
<i>Pinecone (Vector DB)</i>	<1 GB vectors, low query volume (starter tier)	₹ 0.00	Standard plan (higher read/write units)	~₹4,500
<i>Upstash Redis (Cache)</i>	~1K commands/day (within free tier)	₹ 0.00	~200K–300K commands/month	~₹100–200
<i>Object Storage (GCS)</i>	<100 MB resume storage (free tier)	₹ 0.00	~2–3 GB storage	~₹100–200
<i>Backend Hosting (GCP VM / Cloud Run)</i>	Free trial credits / Always Free usage	₹ 0.00	Always-on backend instance	~₹500–800
<i>Frontend Hosting (Vercel)</i>	Hobby tier	₹ 0.00	Same / slightly scaled usage	₹ 0.00
<i>CI/CD (GitHub Actions)</i>	Within free limits	₹ 0.00	Within free limits	₹ 0.00
<i>Total / Month</i>	-	₹ 0	-	~₹8,500 – ₹9,800

The system operates at zero cost during initial deployment by fully leveraging free-tier limits across all services. At moderate scale (~500 users), the architecture sustains production usage at approximately ₹9,000/month, with most of the cost attributed to LLM inference and database scaling, while retaining cost-efficiency through caching and serverless infrastructure.

6. Implementation Timeline (4 Weeks)

The project follows a phased execution strategy, progressing from knowledge base setup and extraction to agent orchestration, frontend integration & final system validation. Each week delivers a fully functional subsystem, ensuring incremental integration and a stable end-to-end pipeline.

	April 2026																													
	W1				W2							W3							W4							W5				
Task/Phase	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Task 1: Project Setup and Core Infrastructure																														
Setup frontend (Next.js) and backend (FastAPI)																														
Configure Supabase (DB + Auth) and Pinecone																														
Setup object storage (GCS) and project repo																														
Task 2: Resume Ingestion and Extraction																														
Implement resume upload and storage																														
Parse PDF and extract structured JSON																														
Add schema validation and error handling																														
Task 3: Core Intelligence Pipeline																														
Build Gap Analysis Agent (Pinecone RAG)																														
Implement Deep Researcher Agent																														
Integrate LLM routing (Gemini + fallback)																														
Task 4: Job Agent and Caching																														
Integrate single job website source																														
Implement job filtering and ranking																														
Add Redis caching for outputs																														
Task 5: Frontend Integration and Deployment																														
Build dashboard UI (profile, gaps, roadmap)																														
Connect frontend with backend APIs																														
End-to-end testing and deployment																														

7. Feasibility Assessment

7.1 Technical Feasibility

<i>Component</i>	<i>Role in Workflow</i>	<i>Feasibility</i>
<i>Frontend (Next.js 15 on Vercel)</i>	User uploads resume, enters target role & later views analysis, roadmap & job matches.	High
<i>Object Storage</i>	Stores the raw resume PDF before processing.	High
<i>Backend API (FastAPI / VM / Cloud host)</i>	Receives upload, triggers processing & coordinates the full pipeline.	High
<i>Resume Extraction Agent</i>	Parses the resume and extracts structured facts such as skills, education, experience, projects & evidence.	High
<i>Structured Profile Store (PostgreSQL / JSONB)</i>	Stores extracted profile JSON, analysis outputs & pipeline metadata.	High
<i>Gap Analysis Agent</i>	Compares extracted profile against target-role requirements and identifies missing skills and priority gaps.	High
<i>Pinecone Vector DB</i>	Stores role requirements, skill taxonomy, learning resources & job-relevant knowledge for retrieval.	High
<i>Deep Researcher Agent</i>	Uses retrieval + live search summaries to build the roadmap and checklist from current role-market knowledge.	Medium-High
<i>Job Finder Agent</i>	Finds and ranks relevant public jobs based on the user profile, gap profile & role fit.	Medium
<i>LLM Routing Layer (2-3 APIs with rotation)</i>	Routes requests across models to stay within free-tier limits and reduce failure risk.	High
<i>Cache (Redis)</i>	Stores extracted profiles, gap results, roadmap outputs & job match responses.	High

7.2 Risk Register

<i>Risk</i>	<i>Likelihood</i>	<i>Impact</i>	<i>Mitigation</i>
<i>Gemini RPD limit hit during demo (1,500/day)</i>	Low	High	Upstash Redis caches results; demo uses pre-cached profiles
<i>pinecone free tier storage (1 GB) exceeded</i>	Low	Medium	Compress embeddings (int8 quantization); migrate to Pinecone free index
<i>Gemini JSON output malformed on edge cases</i>	Medium	Medium	Strict output schema validation with Pydantic; retry with temperature=0
<i>Cloud Run cold starts affect demo UX</i>	Low	Medium	Set min-instances=1 on Cloud Run; keep backend always warm

8. Conclusion

CareerAtlas represents a scalable GenAI system that unifies profile understanding, retrieval-grounded reasoning and agent-driven planning into cohesive pipeline for personalized skill intelligence. By grounding all decisions in continuously updated job market data & leveraging a modular RAG-agent architecture, the system ensures high relevance, adaptability & extensibility without reliance on brittle static ontologies or costly model fine-tuning. Its layered design, efficient cost profile, and well-defined upgrade paths position it for seamless transition from initial deployment to large-scale usage, while maintaining low latency and operational reliability. As a result, CareerAtlas establishes a robust foundation for *intelligent, data-driven career progression systems* that can evolve with both user profiles and industry demands.